

Coppercloud: Blind Server-Supported RSA Signatures

Nikita Snetkov^{1,2}, Jelizaveta Vakarjuk^{1,2} and Alisa Pankova¹
{nikita.snetkov, jelizaveta.vakarjuk, alisa.pankova}@cyber.ee

¹ Cybernetica AS, Mäealuse 2/1, 12618 Tallinn, Estonia

² Tallinn University of Technology, Akadeemia tee 15a, 12618, Tallinn, Estonia

Abstract. Privacy-preserving technologies are a well-established area of research for the modern digital systems, driven by the regulatory mandates, such as the GDPR, and growing demand for the users' privacy. One example from the different available technologies are blind signatures: a primitive that supports creation of privacy-sensitive applications, starting from secure e-voting and anonymous digital cash to privacy-preserving credentials and identity management systems. In this work, we introduce Coppercloud, a blind server-supported RSA signature scheme designed to enhance privacy in digital identity systems. Coppercloud enables a user to obtain a signature on a message, without revealing its content to the supporting server, while distributing the signing key between the user's device and the supporting server. We formalize the security requirements for blind server-supported signing by defining an ideal functionality, and prove that Coppercloud securely realizes this functionality in the Universal Composability (UC) model.

Keywords: Blind Signatures · Universal Composability · RSA · Server-Supported Signatures

1 Introduction

Privacy-preserving technologies (PPTs) have become increasingly prevalent over the past decade. With the adoption of the General Data Protection Regulation (GDPR), which affected both European and international digital space, more and more companies and public institutions implement various PPTs in their solutions. For example, *homomorphic encryption* [42] is deployed to provide private search over online public data [30] and to protect secrecy of a vote in internet elections [15]; *ring signatures* are used to anonymize transactions in Monero [46]; *searchable encryption* [17] could be applied in credential systems [33] and private gun registries [34].

As interest in privacy grows, blind signatures are standing out as a notable example of PPTs. Proposed by David Chaum [14], blind signature protocol allows user to sign a message under a key of some service provider without revealing a message's content to the service provider. The usage of blind signatures can be illustrated with the following examples – in i-voting to register a eligible voter

preserving their secrecy of the vote [23, 37, 48]; in digital cash and cryptocurrencies to disburse untraceable e-cash [41, 10, 29, 51]; in web browser advertisement for private click measurement [49, 47]; in credential systems to protect private attributes from malicious issuing authorities [45]; for privacy-preserving internet CAPTCHA challenges [19]; for user authorization in iCloud Private Relay without a need to provide account information [52].

1.1 Motivation

In April 2024, European Parliament laid the foundation for the new European Digital Identity Framework ³, with one of its goals to establish development of *digital identity wallets* by EU member states. These wallets enable users to digitally sign and present their credentials, using user-generated digital signatures, to create verifiable proofs of identity. Such wallets could be potentially constructed with the usage of threshold signatures [20] to provide verifiable credential presentation. Among the various techniques for building verifiable credential presentation, threshold signatures [20] offer a robust solution by distribution of the signing key. Recently, one of the proposals for EU digital wallets ⁴ was built based on the Smart-ID, digital signing solution ⁵. Smart-ID is an implementation of Buldas et al. server-supported signature scheme [8], which could be seen as two-party threshold signing with asymmetric roles: one party is a lightweight client, the other is a high-performance server. The server-supported signature scheme is functionally interchangeable with the standard RSA signature, which means that it can be verified by a standard RSA verification algorithm without any modifications. It is worth to notice that Smart-ID has protection against offline dictionary attacks and user's memory cloning. In the proposal, supporting server, participating in the signature generation, learns information about the credential that is being presented. This creates a potential privacy leak. Possible solution to this problem is usage of the blinding to hide message being signed from the supporting server.

1.2 Related Work

Blind signatures [14, 28, 25, 27, 22] and threshold signatures [43, 18, 21, 35, 39] have been extensively studied over the last two decades. Even several protocols for *threshold blind signatures* have been proposed [36, 16, 32, 38] as well, however, those schemes only allow to distribute a signing key between several servers/signing providers, giving no ownership to the user. Such techniques do not give a user ownership (even partial) over the signing key.

Server-supported signing usually could be built either by offloading main computational operations to a trusted servers [2, 7, 6, 26, 9], or by using threshold signing [24, 50, 11, 8, 44]. The latter approach by Buldas et al. [8] is used in

³ <https://eur-lex.europa.eu/eli/reg/2024/1183/oj/eng>

⁴ <https://cyber.ee/resources/news/estonian-digital-identity-wallet-mvp/>

⁵ <https://www.smart-id.com/>

Smart-ID and could be applied in EUDI wallets. However, their protocol does not hide content of the signed message from the participating server. Camenish et al. [11] proposed password-authenticated server-supported protocol **Pass2Sign** which supports blindness against malicious server, however, **Pass2Sign** key generation is done locally on user’s device.

1.3 Contribution

Our main contributions in this work are:

1. We formally define a security notion of the *blind server-supported signing* in Canetti’s universal composability (UC) model [13]. We took existing Snetkov et al. [44] ideal functionality $\mathcal{F}^{\text{gSpl}}$, which captures the security properties of Smart-ID and introduced a new functionality $\mathcal{F}^{\text{BlindSpl}}$, following the ideas of Camenish et al. $\mathcal{F}^{\text{Pass2Sign}}$ for introducing blindness properties to the ideal functionality.
2. We present a new protocol for blind server-supported RSA signing – *Coppercloud*. It allows a user to obtain a valid RSA signature on a message without revealing its content to the supporting server, while also distributing the signing key between the user’s device and the server.
3. We prove that Coppercloud securely realizes ideal functionality $\mathcal{F}^{\text{BlindSpl}}$ in UC model.

2 Preliminaries

2.1 Notation

We present necessary notation for this work in Table 1.

$x \leftarrow \$ X$	Sampling of an element x uniformly at random from a set X
$y \leftarrow z$	Assigning of a value z to a variable y
$\parallel$	1. Concatenation 2. Parallel execution of protocols
$\phi(\cdot)$	Euler’s totient function
$\mathbb{Z}_n^\times$	Multiplicative group modulo n
$\mathbb{P}(\lambda)$	Set of primes of size defined by λ

Table 1. Defined notation

2.2 Universal Composability

Universal composability (UC) framework proposed by Canetti [13] allows to define and prove security of the protocols. The core principle behind the UC

framework is ideal vs. real worlds paradigm. When the protocol is executed in the real world, honest parties run the protocol Π over a network that is controlled by an adversary $\mathcal{A}$. Adversary can also corrupt some parties in the real protocol. To prove that the *real world* protocol Π satisfies certain properties, one would need to define an ideal functionality $\mathcal{F}$. It defines how the system should work in the *ideal world*, where honest parties are modelled as dummy parties, who forward their inputs to this ideal functionality. The ideal world adversary interacts directly with $\mathcal{F}$, which explicitly specifies what the adversary is allowed to do. The inputs to the protocol are provided by the environment $\mathcal{Z}$ and the corresponding outputs are sent to $\mathcal{Z}$. Then, one is required to construct a proof showing that, for any real adversary, there exists an ideal adversary, such that no polynomial time environment could distinguish communication in ideal world with the ideal world adversary and the real execution of the protocol with the real adversary.

2.3 Server-Supported Signatures

Server-supported signature as defined by Buldas et al. [8] could be seen as a threshold signing protocol between two parties: client (controlling some smart device like phone or tablet) and server, with following properties:

- *client’s share key-splitting*: on top of standard distributed key generation, client’s keyshare is *split* between client and server. This is done to prevent offline guessing attacks, as the information stored on the device is not enough to test the correctness of adversary’s guess.
- *security against offline dictionary attacks*: client’s *split* keyshare is generated from a password/PIN and a random bitstring. To guess the password, corrupted client needs to communicate with server and they have limited amount of guesses.
- *clone detection*: server has an ability to detect whether adversary managed to create a copy of client’s memory.

2.4 RSA signing

Our protocol produces so called *multi-prime RSA signatures* [31]. Those are generalization of standard bi-prime RSA, where instead of two primes modulus is a product of $v \geq 3$ primes (for Coppercloud, we set $v = 4$). It should be emphasized that multi-prime RSA signatures remain compatible with standard RSA verification algorithms. For the signature padding scheme, we use RSA-PSS [5].

Definition 1 (Multi-prime RSA-PSS). *RSA Probabilistic Signature Scheme defined by $v \geq 3$ prime numbers is called **multi-prime RSA-PSS**. It consists of the key generation RSAKeyGen , signing RSASign and verification RSAVerify algorithms and supporting padding algorithm PSSEncode (Figure 1).*

RSAGen(e, v, λ)	RSASign(sk, msg)
<pre> 1 : $p_1, \dots, p_v \leftarrow \mathbb{P}(\lambda)$ 2 : $N := p_1 \cdot p_2 \dots p_{v-1} \cdot p_v$ 3 : if $[gcd(e, \phi(N)) \neq 1]$, RESTART 4 : $d := e^{-1} \pmod{\phi(N)}$ 5 : return $sk := \langle N, d \rangle, pk := \langle N, e \rangle$ </pre>	<pre> 1 : $\sigma \leftarrow PSS_Encode(msg)^d \pmod{N}$ 2 : return σ </pre>
RSASign(sk, msg)	RSASign(sk, msg)
<pre> 1 : $p_1, \dots, p_v \leftarrow \mathbb{P}(\lambda)$ 2 : $N := p_1 \cdot p_2 \dots p_{v-1} \cdot p_v$ 3 : if $[gcd(e, \phi(N)) \neq 1]$, RESTART 4 : $d := e^{-1} \pmod{\phi(N)}$ 5 : return $sk := \langle N, d \rangle, pk := \langle N, e \rangle$ </pre>	<pre> 1 : $h \leftarrow H_1(msg, l_h)$ 2 : $salt \leftarrow \{0, 1\}^{l_s}$ 3 : $H \leftarrow H_2(0^{64} h salt)$ 4 : $mask \leftarrow H'(H, l_{em} - l_h - 1)$ 5 : $salt_m \leftarrow salt \oplus mask$ 6 : return $M \leftarrow salt_m H 0xBC$ </pre>

Fig. 1. Multiprime RSA-PSS algorithms

The security of Coppercloud is based on the computational hardness of the one-more-RSA assumption, particularly on the equivalent to it the RSA-ACTI (Alternative Chosen-Target RSA Inversion) problem [3].

Definition 2 (RSA-ACTI [4]). Let $\mathcal{A}$ be an oracle Turing machine. For the security parameter λ , let the experiment $\mathbf{Exp}_{\mathcal{A}}^{\text{rsa-acti}}(\lambda)$ be defined as follows:

RSA key pair generation: The challenger generates an RSA public key (N, e) and secret key d corresponding to the security parameter λ . Define the following oracles:

1. The RSA inversion oracle $\mathcal{O}_{\mathcal{I}}(\cdot, N, d)$ that, on input $y \in \mathbb{Z}_N^*$, returns x such that $x^e \leftarrow y \pmod{N}$; i.e., it returns $y^d \pmod{N}$ where $ed \equiv 1 \pmod{\phi(N)}$.
2. An oracle $\mathcal{O}_{\mathcal{R}}(\cdot, N)$ that, when queried, issues a random RSA inversion challenge point, i.e. a random element of $\mathbb{Z}_N^*$. Let y_i denote the outcome of the i -th such query.

$\mathcal{A}$ is invoked: The challenger invokes $\mathcal{A}^{\mathcal{O}_{\mathcal{I}}(\cdot, N, d), \mathcal{O}_{\mathcal{R}}(\cdot, N)}(N, e)$ and responds to its oracle queries. Eventually, $\mathcal{A}$ terminates.

$\mathcal{A}$'s success criterion: Let ℓ be the number of queries $\mathcal{A}$ issued to $\mathcal{O}_{\mathcal{I}}(\cdot, N, d)$. Let $(y_1, \dots, y_n)$ be the values $\mathcal{A}$ received from $\mathcal{O}_{\mathcal{R}}(\cdot, N)$. Let $(z_1, \dots, z_n)$ be $\mathcal{A}$'s output. For $1 \leq i \leq n$, z_i is correct if $z_i^e \equiv y_i \pmod{N}$. $\mathcal{A}$ is successful if $|\{i : z_i \text{ is correct}\}| \geq \ell + 1$.

Let $\mathbf{Adv}_{\mathcal{A}}^{\text{rsa-acti}}(\lambda)$ denote the probability that $\mathcal{A}$ is successful in $\mathbf{Exp}_{\mathcal{A}}^{\text{rsa-acti}}(\lambda)$. The RSA-ACTI problem is hard if for any probabilistic polynomial-time $\mathcal{A}$, $\mathbf{Adv}_{\mathcal{A}}^{\text{rsa-acti}}(\lambda)$ is negligible.

Initialization: On input (init) from $\mathcal{S}$, set $c_S \leftarrow c_C \leftarrow T \leftarrow 0$; $b_{lq} \leftarrow b_{OK} \leftarrow b_{sk} \leftarrow 1$; $ctr_s \leftarrow 0$; $Ver \leftarrow \perp$. Send (init) to $\mathcal{S}$.

Key Generation: On input (keygen, sid, L , T_0 , PIN) from client and (keygen, sid, T_0) from server, if Ver is already recorded or $PIN \notin \{0, \dots, L-1\}$, ignore this query. Otherwise:

- Send (keygen-init) to adversary $\mathcal{S}$.
- Upon receiving (key, Ver) from $\mathcal{S}$, where Ver is signature verification algorithm, store (Ver, L, PIN, T_0) and send (key, Ver) to both client and server.

Corrupt server: On input (corrupt-server) from $\mathcal{S}$: set $c_S \leftarrow 1$ and send (corrupt-server) to server.

Corrupt client: On input (corrupt-client, ℓ) from $\mathcal{S}$, where $\ell \in \{1, 2, 3\}$:

- If $c_C > \ell$, ignore this query.
- Set $b_{lq} \leftarrow \perp$. If $Ver = \perp$, set $c_C \leftarrow 3$. Otherwise, set $c_C \leftarrow \ell$. Send (corrupt-client, ℓ) to client.

Derived values: $b_{KG} \equiv [Ver \neq \perp]$.

Trigger: If $c_S = c_C = 1$, then set $c_C \leftarrow 2$.

Trigger: If $c_C \geq 2$ and b_{KG} then send (corrupt-pin, PIN) to $\mathcal{S}$.

Forwarding: If $c_C = 3$ [resp. $c_S = 1$], then inputs from client [resp. server] are forwarded to $\mathcal{S}$; and $\mathcal{S}$ determines the outputs that client [resp. server] receives.

Fig. 2. Ideal functionality for blind server-supported signing $\mathcal{F}^{\text{BlindSpl}}$ (1/3)

3 Ideal Functionality for Blind Server-Supported Signatures

The ideal functionality for blind server-supported signing $\mathcal{F}^{\text{BlindSpl}}$ is defined on Figures 2-4. The goal of presented ideal functionality is to define the way that we want the system to work. The ideal functionality is constructed by introducing updates (connected to the blinding properties) to the server-supported signing $\mathcal{F}^{\text{gSpl}}$ functionality defined by Snetkov et al. [44]. These changes are inspired by the functionality $\mathcal{F}_{\text{Pass2Sign}}$ introduced by Camenisch et al. [11].

$\mathcal{F}^{\text{BlindSpl}}$ is designed to model an interaction between two distinct parties acting as a *client* and a *server* and any party acting as a verifier. Those two parties have an ability to sign collaboratively upon the client's request, while any party is able to verify any signature via $\mathcal{F}^{\text{BlindSpl}}$.

Corruption Levels and Internal State (follows $\mathcal{F}^{\text{gSpl}}$ functionality [44]) In real-world server-supported signing, a client party is much more prone to various attacks compared to the server. To capture this, $\mathcal{F}^{\text{BlindSpl}}$ has different client's corruption levels: $c_C = 0$ – an honest party; $c_C = 1$ – an adversary accessed client's encrypted memory; $c_C = 2$ – an adversary accessed client's plaintext memory but has no full control over the client; $c_C = 3$ – an adversary has complete control over the client. We denote the device's memory in between the

Signing by client and server: On input $(\text{sign}, \text{sid}, M, \text{PIN}')$ from the client and $(\text{sign}, \text{sid})$ from the server, ignore unless $b_{\text{KG}} \wedge b_{\text{OK}}$. Otherwise:

- Call `clone-check(1)` and `process-pin(PIN')`. If successful, compute σ using `request-sig(sid, M)`.
- If $\sigma \neq \perp$, output $(\text{sign-success}, \text{sid}, M, \sigma)$ to client, and $(\text{sign-success}, \text{sid})$ to server. Otherwise, return $(\text{sign-fail}, \text{sid})$ to client and server.

Signing by client and adversary: On input $(\text{sign}, \text{sid}, M, \text{PIN}')$ from the client and $(\text{sign-server}, \text{sid})$ from $\mathcal{S}$, ignore unless $b_{\text{KG}} \wedge [\text{cs} = 1]$. Otherwise:

- Call `process-pin(PIN')` and then compute σ by querying `request-sig(sid, M)`.
- If $\sigma \neq \perp$, output $(\text{sign-success}, \text{sid}, M, \sigma)$ to client. Otherwise, return $(\text{sign-fail}, \text{sid})$ to both client and $\mathcal{S}$.

Signing by adversary and server: On input $(\text{sign-client}, \text{sid}, \text{PIN}')$ from $\mathcal{S}$ and $(\text{sign}, \text{sid})$ from the server: ignore unless $b_{\text{KG}} \wedge b_{\text{OK}} \wedge [\text{c}_\text{C} > 0]$. Otherwise go to step $\text{c}_\text{C} \in \{1, 2, 3\}$:

1. Call `clone-check(0)` and `process-pin(PIN')`. If successful, set $\text{c}_\text{C} \leftarrow 2$ and go to step 3.
2. Call `clone-check(0)`. If successful, go to step 3.
3. Send $(\text{sign-init}, \text{sid})$ to $\mathcal{S}$ and upon receiving $(\text{sign-success}, \text{sid})$, output $(\text{sign-success}, \text{sid})$ to server. Set $\text{ctr}_s \leftarrow \text{ctr}_s + 1$. Otherwise, return (sign-fail) to both $\mathcal{S}$ and server.

Verification: On input $(\text{verify}, \text{sid}, M, \sigma)$ from a party P , send $(\text{verify}, \text{sid}, M, \sigma)$ to $\mathcal{S}$. Upon receiving $(\text{verified}, \text{sid}, M, \sigma, \phi)$ from $\mathcal{S}$ with $\phi \in \{1, 0\}$:

- if (M, σ, b') is recorded, then set $b = b'$
- if $\phi = 1$, (M, σ, b) has not been recorded before, and $b_{\text{sk}} = 1$, set $\text{ctr}_s \leftarrow \text{ctr}_s - 1$:
 - if $\text{ctr}_s \geq 0$, record $(M, \sigma, b = \phi)$. // saving valid signature message pairs that have not been saved during "signing by adversary and server" queries
 - if $\text{ctr}_s < 0$, set $b = 0$ and record (M, σ, b) // (one-more unforgeability) the adversary created $n + 1$ valid signatures with n signing queries
- else, set $b = \phi$ and record (M, σ, b) .
- output $(\text{verified}, \text{sid}, M, \sigma, b)$.

Fig. 3. Ideal functionality for blind server-supported signing $\mathcal{F}^{\text{BlindSpl}}$ (2/3)

signing sessions as encrypted memory and the device's memory during the signing session as plaintext memory.

Adversary could "switch" between different corruption levels which corresponds to real-life scenarios and attacks. For example, let us assume an adversary, who copied client's encrypted memory, manages to guess user's PIN and decrypts the client's memory. This case in $\mathcal{F}^{\text{BlindSpl}}$ is modelled as adversary increasing its corruption level from $\text{c}_\text{C} = 1$ to $\text{c}_\text{C} = 2$).

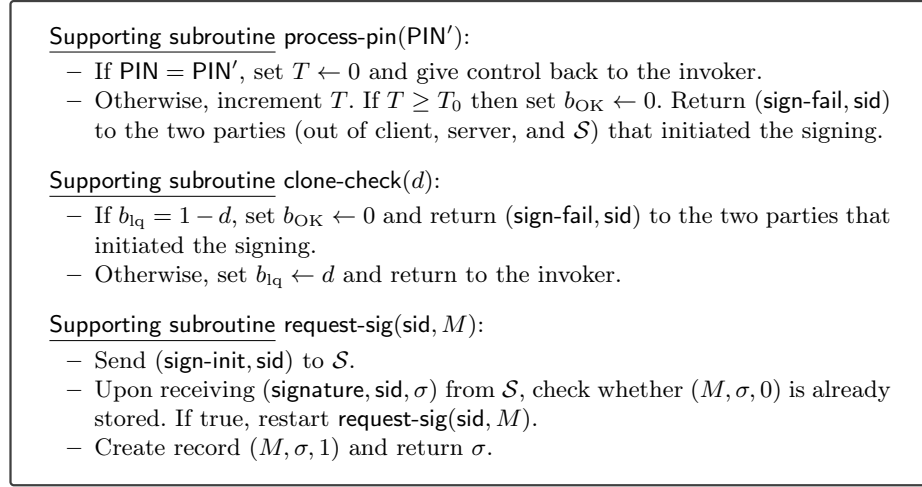


Fig. 4. Ideal functionality for blind server-supported signing $\mathcal{F}^{\text{BlindSpl}}$ (3/3)

Server's corruption levels are more straightforward: $c_C = 0$ represents *honest server's* case, $c_S = 1$ – full corruption of the server. In some sense, we model static adversary for the server and adaptive for the client.

To track different attack scenarios, internal state of the ideal functionality $\mathcal{F}^{\text{BlindSpl}}$ is initialized with the set of following flags:

- b_{OK} – governs the signing capability of $\mathcal{F}^{\text{BlindSpl}}$. If $b_{\text{OK}} = 0$, functionality will ignore any signing requests.
- b_{lq} – tracks origin of the last signing query. $b_{\text{lq}} = 1$ corresponds to the most recent query made by client, $b_{\text{lq}} = 0$ – by adversary.
- b_{sk} – indicates whether the adversary knows (or is assumed to know) the total private key. If $b_{\text{sk}} = 0$, adversary managed to corrupt both server and client at any positive corruption level, therefore $\mathcal{F}^{\text{BlindSpl}}$ cannot provide any security guarantees.
- b_{KG} – tracks whether `keygen` query was made to ideal functionality.
- ctr_s – tracks number of successful signing sessions by the adversary and server to save only this number of valid signatures during verification

PIN/Password Handling (follows $\mathcal{F}^{\text{Spl}}$ functionality [44]) Since the client's key share is generated using some low-entropy password or PIN, $\mathcal{F}^{\text{BlindSpl}}$ has supporting subroutine `process-pin`. It verifies a PIN value of length L provided by client or adversary. To model protection against password offline-guessing, $\mathcal{F}^{\text{BlindSpl}}$ tracks a number of unsuccessful PIN queries with the counter T . Whenever T becomes greater or equal than the allowed number of attempts T_0 , $\mathcal{F}^{\text{BlindSpl}}$ will ignore any signing request and assign $b_{\text{OK}} = 0$.

Clone detection (follows $\mathcal{F}^{\text{Spl}}$ functionality [44]) To model server's functionality to detect and prevent illegitimate copies of the client's memory, $\mathcal{F}^{\text{BlindSpl}}$ has

a supporting subroutine `clone-check`. This subroutine checks if the previous signing query came from the same source. If not, $\mathcal{F}^{\text{BlindSpl}}$ sets $b_{\text{OK}} = 0$.

Blind Signing We define three separate signing interfaces in the functionality, for different corruptions. Signing by client and server is initiated by both parties inputs, where the message M and PIN are the client's inputs. Next, the functionality verifies the source of the previous signing query (clone detection) and the PIN. If both checks are successful, signature σ is requested from the adversary, without giving M as input (ensuring blindness). The message signature pair is recorded by the functionality. Signing by client and adversary is similar to the previously described interface. The only difference is that there is no clone detection verification. The reason is that the malicious server should be responsible for running this check and we cannot expect adversary to run it correctly. Finally, there is signing by adversary and the server interface, which depends on the client's level of corruption. For the first corruption level (client's encrypted memory), functionality verifies the source of last signing query and the PIN' correctness. If both checks are successful, corruption level increases. For the second corruption level (client's plaintext memory but no full control over the client), only the source of last signing query is verified, as in this case we assume adversary already knows correct PIN. For the third corruption level no additional checks are performed as adversary has full access over the client. In all three cases, functionality proceeds by asking adversary to create σ . For the case where client is malicious, we do not save signature message pairs to the functionality during the signing. Since the adversary may just ignore M input and choose something else instead during the signing. We keep a counter of successful signing sessions ctr_s . Next, we save pairs during the verification procedure and for signatures generated by the adversary (the ones that have not been yet saved to the functionality), we let the adversary decide the outcome of the verification. For unforgeability, we need to make sure that we save only the ctr_s valid pairs as this was the number of successful signing sessions.

Verification Verification corresponds to the standard way of performing verification [12]. The verification request is performed through the adversary. For the signatures that have not been recorded by the functionality, adversary decides what the outcome of the verification will be. For the signatures created with the honest client, the functionality decides the verification outcome as it has (M, σ, b) already recorded. For the malicious client case, we let the adversary decide the outcome of verification as the signature itself was created by adversary. Additionally, for the signatures created solely by the adversary, we let the adversary decide the verification outcome.

4 Coppercloud: Blind Server-Supported RSA signing

Coppercloud is a two party protocol between a client and a server which allows them to collaboratively generate RSA signing key and create RSA signatures

under this key. Signing has a blinding mechanism identical to RSA-BSSA [40], which hides message's content from the supporting server. The contribution of Coppercloud in combining the blindness mechanism of RSA-BSSA with the properties of Server-Supported signatures, defined in Section 3.

Key Generation Key generation protocol is the same as defined by Snetkov et al. [44]. The key generation protocol is defined on Figure 5.

As an input both parties receive RSA public exponent e and security parameter λ . As additional input server receives a number of attempts T_0 . Client samples two prime numbers (corresponding to λ -bit security) and computes modulus such that $n_1 \leftarrow p_1 \cdot q_1$. Next, they compute a private exponent $d_1 \leftarrow e^{-1} \bmod \phi(n_1)$ and generate a bitstring $u \leftarrow \{0, 1\}^\lambda$. Based on PIN value submitted by the user, client calculates two additive shares as $d'_1 \leftarrow \text{genShare}(u, \text{PIN}, n_1)$ (Algorithm 1) and $d''_1 \leftarrow d_1 - d'_1$. Finally, client sends $\langle n_1, d'_1 \rangle$ to the server. We need to ensure that the value d'_1 that can be derived through trying out different PINs is not verifiable under pk_1 . Since it is only one share of the corresponding private key and the second share is stored on the server, we ensure protection against offline guessing attacks.

Parallel to the client's execution, server samples two primes p_2, q_2 and computes modulus n_2 . Upon receiving $\langle n_1, d'_1 \rangle$, they compute total modulus $n \leftarrow n_1 \cdot n_2$ and clone detection bitstring w . Clone detection bitstring is needed to track the source of the signing query. If the client's memory gets leaked and the adversary and the client try to initiate signing section with the same string w , it will be detected by the server. Then server transfers $\langle n, w \rangle$ to the client. Finally, the server initialise number of failed attempts as $T \leftarrow 0$ and store $n, n_1, n_2, d''_1, d_2, w, T, T_0$.

Client, upon obtaining $\langle n, w \rangle$, verifies if the total modulus n is divisible by n_1 . Client aborts the key generation on the failure, otherwise securely deletes all the values except n, n_1, u, w . Public key is defined as $\text{pk} \leftarrow (n, e)$.

Inputs: The maximum length of the output size, password PIN and AES encryption function $\text{AES_Enc}_{\text{Key}}(\cdot)$:

```

for  $j \in \{0, 1, 2, \dots, 255\}$  do
   $x \leftarrow \text{AES\_Enc}_{\text{PIN}}(u \parallel j)$ 
  if  $x < \text{size}$  then
    return  $x$ ;
  end
end
return  $\perp$ ;

```

Algorithm 1: Supporting algorithm for key splitting $\text{genShare}(u, \text{PIN}, \text{size})$

$\text{KeyGen}_C(\lambda, e)$	$\text{KeyGen}_S(\lambda, e, T_0)$
1 : generate primes p_1, q_1 parametrised by λ	1 : generate primes p_2, q_2 parametrised by λ
2 : $n_1 \leftarrow p_1 \cdot q_1$	2 : $n_2 \leftarrow p_2 \cdot q_2$
3 : $d_1 \leftarrow e^{-1} \bmod \phi(n_1)$	3 : $d_2 \leftarrow e^{-1} \bmod \phi(n_1)$
4 : $u \leftarrow \{0, 1\}^\lambda$	4 : $C \Rightarrow: \langle d_1'', n_1 \rangle$
5 : $d_1' \leftarrow \text{genShare}(u, \text{PIN}, n_1)$	5 : $n \leftarrow n_1 \cdot n_2$
6 : $d_1'' \leftarrow (d_1 - d_1') \bmod \phi(n_1)$	6 : $w \leftarrow \{0, 1\}^\lambda$
7 : $\Rightarrow S : \langle d_1'', n_1 \rangle$	7 : $\Rightarrow C : \langle n, w \rangle$
8 : $S \Rightarrow: \langle n, w \rangle$	8 : $T = 0$
9 : If $n_1 \nmid n$, ABORT	9 : store $\langle n, n_1, n_2, d_1'', d_2, w, T, T_0 \rangle$
10 : $w' \leftarrow w$	10 : return $\text{pk} \leftarrow \langle n, e \rangle$
11 : store $\langle n, n_1, u, w' \rangle$	
12 : return $\text{pk} \leftarrow \langle n, e \rangle$	

Fig. 5. Coppercloud Key Generation Protocol

Signing The signing protocol is defined on Figure 6. The client starts with deriving key share d_1' using PIN supplied by the user. Next, the client blinds encoded message msg to receive m_* . Blinded message is used to derive partial client's signature share y . The message consisting of the client's partial signature y , blinded message m_* and the clone-detection bitstring w' , is sent to the server.

Upon receiving client's message, server verifies clone-detection bitstring. Next, the server completes client's signature share σ_1 using stored share d_1'' and verifies received signature. Successful verification of the client's signature share shows to the server that the user entered correct PIN (and the correct d_1' was derived). The server proceeds with generating their signature share σ_2 and combining both shares using Chinese Remainder Theorem (CRT) to receive σ_* . Next, the server generates and updates clone-detection bitstring and sends it together with the signature σ_* to the client.

On message from the server, client unblinds received signature and verifies the resulting signature σ . If the signature verifies, the client outputs σ . Client saves new clone detection bitstring w .

Verification RSA signatures produced by Coppercloud pass standard RSA-PSS verification algorithm RSASVerify (Figure 1).

Efficiency Introduced changes to Buldas et al [8] protocol do not influence key generation. Blinding operation brings a negligible overhead to signing operation and only affects client's execution [1].

$\text{Blind}(M)$	$\text{UnBlind}(\sigma_*, r, n)$
1 : $r \leftarrow \mathbb{Z}_{n_1}^\times \wedge \gcd(r, n_1) = 1$	1 : $\sigma \leftarrow \sigma_* \cdot r^{-1} \bmod n$
2 : return $(r, M \cdot r^e \bmod n_1)$	2 : return σ
$\text{Sign}_C(\text{msg}, \text{PIN}, u, w', n_1, n)$	$\text{Sign}_S(d_2, d_1'', n_2)$
1 : $d_1' \leftarrow \text{genShare}(u, \text{PIN}, n_1)$	1 : $C \Rightarrow: \langle y, m_*, w' \rangle$
2 : $r, m_* \leftarrow \text{Blind}(\text{PSSEncode}(\text{msg}))$	2 : check $w' = w \wedge T < T_0$
3 : $y \leftarrow (m_*)^{d_1'} \bmod n_1$	3 : $\sigma_1 \leftarrow m_*^{d_1''} \cdot y \bmod n_1$
4 : $\Rightarrow S : \langle y, m_*, w' \rangle$	4 : if $\sigma_1^e \not\equiv m_* \pmod{n_1}$ then
5 : $S \Rightarrow: \langle \sigma_*, w \rangle$	5 : $T \leftarrow T + 1$ and stop
6 : $w' \leftarrow w$	6 : $\sigma_* \leftarrow \text{CRT}_{n_1, n_2}(\sigma_1, m_*^{d_2} \bmod n_2)$
7 : $\sigma \leftarrow \text{UnBlind}(\sigma_*, r, n)$	7 : update $w \leftarrow \{0, 1\}^\lambda$
8 : return σ	8 : $T \leftarrow 0$
	9 : $\Rightarrow C : \langle \sigma_*, w \rangle$

Fig. 6. Coppercloud Signing Protocol

5 Security

5.1 Correctness

Let us prove the correctness of composition operation for blind signatures:

According to the signing protocol (Figure 6), $m_* = M \cdot r^e \bmod n_1$, $\sigma_1 = m_*^{d_1'} \cdot y = m_*^{d_1} \bmod n_1$ and $\sigma_2 = m_*^{d_2} \bmod n_2$. Chinese Remainder Theorem allows to compose both signatures under the same modulus $n = n_1 \cdot n_2$ as $\sigma_* = \text{CRT}(\sigma_1, \sigma_2) = \sigma_1 \cdot n_2 \cdot y_2 + \sigma_2 \cdot n_1 \cdot y_1 \bmod n$, where $y_i = n^{-1} \bmod n_{3-i}$. Then, $\sigma_1 \cdot n_2 \cdot y_2 + \sigma_2 \cdot n_1 \cdot y_1 = M^{d_1} \cdot r^{ed} \cdot n_2 \cdot y_2 + M^{d_2} \cdot r^{ed} \cdot n_1 \cdot y_1 \bmod n = r \cdot (M^{d_1} \cdot n_2 \cdot y_2 + M^{d_2} \cdot n_1 \cdot y_1) \bmod n = r \cdot \sigma$

Therefore, Coppercloud's CRT composition calculation is correct. $\square$

5.2 Security proof

Since our goal is to prove the security of the blinded scheme, we intentionally prevent the supporting server from learning the content of the messages it helps to sign. Therefore, we are not expecting adversary to be able to forge a signature on a previously unsigned message, as the server does not know the messages it has already signed. Instead, we want to achieve one-more unforgeability, where $\mathcal{A}$ (corrupted client) interacts with the server in n blind signing sessions. The adversary succeeds in winning if it can produce $n+1$ valid and distinct message-signature pairs that pass the verification. Therefore, we have to rely on one-more RSA assumption instead of RSA assumption. Since RSA-ACTII is equivalent to one-more RSA assumption, our security proof relies on that.

Theorem 1. *If genShare is a pseudo-random function then protocol $\pi_{\text{Coppercloud}}$ presented in Figures 5-6 is a secure implementation of $\mathcal{F}^{\text{BlindSpl}}$ in the random oracle model under the alternative chosen-target RSA inversion (RSA-ACTI) problem.*

We construct a simulator Sim , such that for any adversary $\mathcal{A}$, no environment cannot distinguish an execution with $\pi_{\text{Coppercloud}} \parallel \mathcal{A}$ from an execution with $\mathcal{F}^{\text{BlindSpl}} \parallel \text{Sim} \parallel \mathcal{A}$. Simulator state consists of $\langle \mathbf{c}_S, \mathbf{c}_C, n, n_1, n_2, d_1, d'_1, d''_1, d_2, u, w \rangle$. Next we describe how the simulator handles and responds to the commands from $\mathcal{F}^{\text{BlindSpl}}$. In the proof, we assume static corruptions for the server and adaptive corruptions for the client. We divide description of the simulator on the corrupted/honest server sections.

The main idea for the proof is to receive challenge for RSA-ACTI problem pk^* and if the server is honest, simulator uses pk^* as honest server's public key share $\text{pk}_2 \leftarrow \text{pk}^*$. For the signing sessions initiated through the $\mathcal{F}^{\text{BlindSpl}}$, simulator simulates the server's part of the protocol without knowing message and using access to the RSA inversion oracle $\mathcal{O}_{\mathcal{I}}(\cdot)$. When answering RO queries for encoding, simulator embeds RSA challenge y_i in the response. Once $\mathcal{A}$ queries verification on (M, σ) , if the signature verifies and has not been created while communicating to the simulated server, the RSA challenge is embedded to the message signed but $\mathcal{O}_{\mathcal{I}}(\cdot)$ has not been queried on this input. Hence, simulator can extract answer to the RSA-ACTI challenge from (M, σ) .

If the server is corrupted, simulator uses pk^* as honest client's public key share $\text{pk}_1 \leftarrow \text{pk}^*$. For the signing sessions, simulator does not get message M as input, it samples random signature $\sigma \leftarrow \$ \mathbb{Z}_n$ and later programs RO to make sure that the encoded message $M \leftarrow \sigma^e \pmod n$ is mapped to the actual message that was signed msg . Programming of the RO happens when simulator receives query with msg as input.

Proof. We define the simulator as follows.

Honest server The simulator gets input $\text{pk}^* = (N, e)$ from the external challenger and simulates the protocol run as described below.

Corruption: On command “Corrupt client” from $\mathcal{A}$ before the key generation has started, send $(\text{corrupt-client}, 3)$ to $\mathcal{F}^{\text{BlindSpl}}$.

For the key generation, simulator uses challenge public key pk^* as the public key share of the server and generating the values of the public key share of the other party.

Key generation: On command (keygen-init) from $\mathcal{F}^{\text{BlindSpl}}$:

- If the client is honest, simulate both client and server. Set $\text{pk}_2 \leftarrow \text{pk}^*$, initialise values n_1, d_1, n, u, w according to the protocol. Generate $d''_1 \leftarrow \$ \mathbb{Z}_{\phi(n_1)}$. The value d'_1 does not get explicitly defined (but it is defined through d_1 and d''_1) and value d_2 is available in the form of oracle access $\mathcal{O}_{\mathcal{I}}(\cdot)$. Set $\text{pk} \leftarrow (n, e)$ and $\text{Ver} \leftarrow \text{pk}$ and send (key, Ver) to $\mathcal{F}^{\text{BlindSpl}}$.
 // The real protocol execution is different from the simulated one in the way how d''_1 is generated. In the real protocol, $d''_1 \leftarrow d_1 - d'_1 \pmod{\phi(n_1)}$, where $d_1 \leftarrow e^{-1}$

$(\text{mod } n_1)$ and $d'_1 \leftarrow \text{genShare}(\text{PIN}, u)$. In the simulated, $d''_1 \leftarrow \mathbb{Z}_{\phi(n_1)}$. Since genShare algorithm relies on a PRF which is a bijection, the distribution of the value d'_1 in both cases is identical.

- If the client is corrupted, simulate the server's protocol execution:
 - Set $\text{pk}_2 \leftarrow \text{pk}^*$.
 - Upon receiving $\langle n_1, d''_1 \rangle$ from $\mathcal{A}$, compute $n \leftarrow n_1 \cdot n_2$, sample clone detection bitstring $w \leftarrow \{0, 1\}^*$ and send $\langle n, w \rangle$ to $\mathcal{A}$.
 - Set $\text{pk} \leftarrow (n, e)$ and $\text{Ver} \leftarrow \text{pk}$ and send (key, Ver) to $\mathcal{F}^{\text{BlindSpl}}$.
 - Since corrupting client during key generation corresponds to full corruption $c_C = 3$, receive also $(\text{corrupt-pin}, \text{PIN})$ from $\mathcal{F}^{\text{BlindSpl}}$.

The values d_1, d'_1, u remain undefined and d_2 is available in the form of oracle access $\mathcal{O}_I(\cdot)$.

// The real protocol execution is identical to the simulated one, since all the values are generated in the same way as in the real protocol.

Corruption (after key generation):

- On command “Leak encrypted memory” from $\mathcal{A}$, ignore this query if the keys have not yet been generated. Otherwise, send $(\text{corrupt-client}, 1)$ to $\mathcal{F}^{\text{BlindSpl}}$ and $\langle w, n_1 \rangle$ to the $\mathcal{A}$. The value u has been initialised during the key generation, send it to $\mathcal{A}$.
- On command “Leak unencrypted memory” from $\mathcal{A}$, ignore this query if keys have not yet been generated. Otherwise, send $(\text{corrupt-client}, 2)$ to $\mathcal{F}^{\text{BlindSpl}}$ and $\langle w, n_1 \rangle$ to the $\mathcal{A}$. Receive PIN from $\mathcal{F}^{\text{BlindSpl}}$ and compute d'_1 from PIN and u using genShare and send $\langle d'_1, u \rangle$ to $\mathcal{A}$.
- On command “Corrupt client” from $\mathcal{A}$, if keys have been generated, proceed in the same way as when responding to “Leak unencrypted memory” command. Additionally, redefine $d''_1 \leftarrow d_1 - d'_1$ for d'_1 computed from PIN and u . // this is needed to ensure correctness of the signatures created during the signing queries. Otherwise, send $(\text{corrupt-client}, 3)$ to $\mathcal{F}^{\text{BlindSpl}}$.

Signing:

On command “Signing by client and server”, simulate both client and server without knowing msg , since nobody is corrupted. On input $(\text{sign-init}, \text{sid})$ from $\mathcal{F}^{\text{BlindSpl}}$:

- Generate random $\text{salt} \leftarrow \$ \{0, 1\}^{l_s}$ and follow steps of PSSEncode (Figure 7) to produce encoded message M , challenge y_i and randomness r_i .
- Compute blinded message $(m_*, r) \leftarrow \text{Blind}(M)$, client's signature share $\sigma_1 \leftarrow m_*^{d_1}$ and set $y \leftarrow \sigma_1 \cdot m_*^{-d''_1} \text{ mod } n_1$.
- Simulate sending $\langle y, m_*, w \rangle$ to server.
- Compute $\sigma_1 \leftarrow y \cdot m_*^{d''_1} \text{ mod } n_1$, get server's signature share σ_2 by querying oracle $\mathcal{O}_I(m_*)$. Combine signature shares $\sigma_* \leftarrow \text{CRT}(\sigma_1, \sigma_2)$.
- Simulate sending $\langle \sigma_*, w', m_* \rangle$ to client.
- Unblind signature $\sigma \leftarrow \text{UnBlind}(\sigma_*, r)$ and set $z_i \leftarrow \sigma / r_i$ and save record (y_i, z_i) in $\mathbf{T}_{\text{RSA}}$.

- Send command $(\text{signature}, \text{sid}, \sigma)$ to $\mathcal{F}^{\text{BlindSpl}}$ and save partial record $(\text{sid}, \perp_m, \sigma, M, \text{salt})$.
 // In the real protocol, $y \leftarrow m_*^{d'_1}$. In the simulated, $y \leftarrow \sigma_1 \cdot m_*^{-d''_1}$, where $\sigma_1 \leftarrow m_*^{d_1}$. In the real protocol, $d'_1 \leftarrow \text{genShare}(\text{PIN}, u)$ and in the simulated protocol $d'_1 = d_1 - d''_1$ since $y = m_*^{d_1} \cdot m_*^{-d''_1} = m_*^{d_1 - d''_1}$. Since genShare algorithm relies on a PRF which is a bijection, the distribution of the value d'_1 in both cases is identical. Therefore, the distribution of y is identical in both cases.

On command “Signing by adversary and server”, simulate server’s protocol execution:

- With the first message from $\mathcal{A}$, receive (σ'_1, w, m_*) and simulate constructing command $(\text{sign}, \text{sid}, \text{PIN}')$ to $\mathcal{F}^{\text{BlindSpl}}$ depending on the corruption level:
 - If encrypted memory is leaked ($c_C = 1$), simulator has already initialised bitstring’s u value. With the message from $\mathcal{A}$, receive $\sigma'_1 = m_*^{d'_1} \bmod n_1$. Having u , test through all possible $\text{PIN} \in L$ to find PIN' that produces σ'_1 from $d'_1 = \text{genShare}(u, \text{PIN})$. Set $\text{PIN}' = \text{PIN}$ and send command $(\text{sign}, \text{sid}, \text{PIN}')$ to $\mathcal{F}^{\text{BlindSpl}}$.
 - If unencrypted memory is leaked $c_C = 2$, simulator has already initialised value u and knows PIN . Set $\text{PIN}' = \text{PIN}$ and send command $(\text{sign}, \text{sid}, \text{PIN}')$ to $\mathcal{F}^{\text{BlindSpl}}$.
 - If client is fully corrupted $c_C = 3$, simulator already knows PIN . Set $\text{PIN}' = \text{PIN}$ and send command $(\text{sign}, \text{sid}, \text{PIN}')$ to $\mathcal{F}^{\text{BlindSpl}}$.
- Upon command $(\text{sign-init}, \text{sid})$ from $\mathcal{F}^{\text{BlindSpl}}$, compute $\sigma_1 = \sigma'_1 \cdot m_*^{d''_1} \bmod n_1$ using stored d''_1 and $\sigma_2 = m_*^{d_2} \bmod n_2$ by querying $\mathcal{O}_{\mathcal{I}}(m_*)$.
- Combine σ_1 and σ_2 to the final (blinded) signature σ_* using Chinese Remainder Theorem.
- Verify σ_* , if it is correct, send command $(\text{sign-success}, \text{sid})$ to $\mathcal{F}^{\text{BlindSpl}}$.
 // The real protocol execution is identical to the simulated one, since all values are generated in the same way as in the real protocol.

The simulator makes call to the RSA-ACTI oracle $\mathcal{O}_{\mathcal{I}}(\cdot)$ for each of the signing sessions initiated by the adversary (malicious client).

Verification: If the client is corrupted, functionality has not recorded any (M, σ) pairs. The correct pairs can be recorded in the functionality during the verification procedure. So far, simulator has embedded RSA challenge y_i in each signature (through the RO). Once $\mathcal{A}$ queries verification on (M, σ) , if the signature verifies and has not been created while communicating to the simulated server, the RSA challenge is embedded to the signature but the oracle $\mathcal{O}_{\mathcal{I}}(\cdot)$ has not been queried on this input. Hence, simulator can extract answer to the RSA-ACTI challenge from (M, σ) .

- Upon receiving message $(\text{verify}, \text{sid}, M, \sigma)$ from $\mathcal{F}^{\text{BlindSpl}}$, verify signature using stored $\text{pk} = (e, n)$ following the instructions from Algorithm 1.
- If the signature is correct, set $\phi \leftarrow 1$. Otherwise, set $\phi \leftarrow 0$.
 // at this point, it can be either signature created with the help of simulator or by the adversary alone

On input to oracle H_2 of the form $v_i = (0^{64} \| h_i \| \text{salt}_i)$, if there is no H_i defined proceed as: // embedding RSA challenge to RO responses

1. Get RSA challenge $y_i \leftarrow \mathcal{O}_{\mathcal{R}}(\cdot)$
2. Find $r_i \in \mathbb{Z}_N$ such that, for $w_i = y_i \cdot r_i^e$, $w_i < 2^{k-1}$ and the binary representation of w_i ends in the byte $0xBC$
3. Set $M \leftarrow w_i$ and parse $w_i = \text{salt}_{m_i} \| H_i \| 0xBC$
4. Set $DB_i \leftarrow 0^a \| 0x01 \| \text{salt}_i$ and let $\text{mask} \leftarrow DB_i \oplus \text{salt}_{m_i}$
5. Program RO H' such that $H'(H_i, l) \leftarrow \text{mask}$. If the value is already defined, fail.
6. Set $H_2(v_i, l_h) \leftarrow H_i$, save (i, v_i, r_i, w_i, H_i) and output H_i

Fig. 7. Answering H_2 queries of the form $v_i = (0^{64} \| h_i \| \text{salt}_i)$

On input to oracle H_1 of the form (msg, l) : // making sure that simulated signatures verify correctly

1. If there is record for $h_i = (\text{msg}, l)$, proceed with h_i . Else, sample $h_i \leftarrow \$\{0, 1\}^l$.
2. Find non-completed records of the form $(\text{sid}, \perp_{\text{msg}}, \sigma_i, M_i, \text{salt}_i)$.
3. For all i , send $(\text{verify}, \text{sid}, \text{msg}, \sigma_i)$ to $\mathcal{F}^{\text{BlindSpl}}$.
4. On message $(\text{verify}, \text{sid}, \text{msg}, \sigma_i)$, respond with $(\text{verified}, \text{sid}, \text{msg}, \sigma_i, \phi \leftarrow 0)$. Upon receiving $(\text{verified}, \text{sid}, \text{msg}, \sigma_i, b \leftarrow 1)$ for some i (meaning that the message-signature pair has been recorded by the functionality), parse $M_i = \text{salt}_{m_i} \| H_i \| 0xBC$.
5. If there is no such i (the message has not yet been signed), set $h_i \leftarrow H_1(\text{msg}, l)$ and output h_i . // only for honest client and honest server case
6. Else, if h_i has not been defined yet, set $h_i \leftarrow H_1(\text{msg}, l)$. Program RO by setting $H \leftarrow H_2(0^{64} \| h_i \| \text{salt})$. Complete signing record by storing msg and output h_i .

Fig. 8. Answering H_1 queries of the form (msg, l) .

- Find i such that $\sigma^e \bmod N \bmod n_1 \equiv w_i$. Fail if such i does not exist.
- Since $w_i = y_i \cdot r_i^e \bmod n_1$, $y_i = (\sigma/r_i)^e$. Set $z_i \leftarrow \sigma/r_i$.
// meaning that inverse for challenge y_i is found
- Store $(y_i, z_i) \in \mathbb{T}_{\text{RSA}}$.
- Let q_R be number of queries simulator made to the challenge oracle $\mathcal{O}_{\mathcal{R}}()$ and $q_I \leq q_R - 1$ be number of queries simulator made to the inversion oracle $\mathcal{O}_{\mathcal{I}}()$. If number of entries in $\mathbb{T}_{\text{RSA}}$ is q_R , output all $z_i \in \mathbb{T}_{\text{RSA}}$ to the external challenger as solution to RSA-ACTI problem.
// that is, simulator made less queries to the inversion oracle than to the challenge oracle and managed to find solution to all the received challenges
- Output $(\text{verified}, \text{sid}, M, \sigma, \phi)$ to $\mathcal{F}^{\text{BlindSpl}}$.
// The difference between the real and ideal protocol execution is when the simulated verification returns $b = 0$ while the real verification algorithm returns $b = 1$. The only verification query with correct signature to which the simulator responds with 0 is the one that constitutes finding solution to RSA-ACTI problem. This, in turn, means that the adversary made q_I signing queries through the functionality, but managed to produce $q_I + 1$ valid signatures. The real protocol execution will return 1 on verification

of $n + 1$ -th valid signature and the simulated execution returns 0. If adversary manages to differentiate ideal and the real execution, it is able to solve RSA-ACTI problem.

Corrupted server The simulator gets input $\text{pk}^* = (N, e)$ from the external challenger and simulates the protocol run as described below. For the key generation, simulator uses challenge public key pk^* as the public key share of the client and generating the values of the public key share of the other party.

Corruption: On command “Corrupt client” from $\mathcal{A}$ before the key generation has started, send $(\text{corrupt-client}, 3)$ to $\mathcal{F}^{\text{BlindSpl}}$. In this case, the adversary $\mathcal{A}$ knows full private key $\langle d, n \rangle$ and can create valid signatures to the messages of its choice. Thus, our scheme does not provide any security guarantees under this setup.

Key generation: On command (keygen-init) from $\mathcal{F}^{\text{BlindSpl}}$:

- If client is honest, simulate the client’s protocol execution:
 - Set $\text{pk}_1 \leftarrow \text{pk}^*$ and sample randomly $d_1'' \leftarrow \mathbb{Z}_{n_1}$.
 - Send $\langle d_1'', n_1 \rangle$ to $\mathcal{A}$.
 - Upon receiving $\langle n, w \rangle$ from $\mathcal{A}$, check whether $n_1 \mid n$, set $\text{pk} \leftarrow (n, e)$ and $\text{Ver} \leftarrow \text{pk}$. Send (key, Ver) to $\mathcal{F}^{\text{BlindSpl}}$.

The values d_1', d_2, u, n_2 remain undefined and d_1 is available in the form of oracle access $\mathcal{O}_{\mathcal{I}}(\cdot)$.

// The real protocol execution is different from the simulated one in the way how d_1'' is generated. In the real protocol, $d_1'' \leftarrow d_1 - d_1' \pmod{\phi(n_1)}$, where $d_1 \leftarrow e^{-1} \pmod{\phi(n_1)}$ and $d_1' \leftarrow \text{genShare}(\text{PIN}, u)$. In the simulated, $d_1'' \leftarrow \mathbb{Z}_{n_1}$. Since genShare algorithm relies on a PRF which is a bijection, the distribution of the value d_1'' in both cases is identical.

- If client is corrupted, then the adversary $\mathcal{A}$ knows full private key $\langle d, n \rangle$ and can create valid signatures to the messages of its choice. Thus, our scheme does not provide any security guarantees under this setup.

Corruption (after key generation):

- On command “Leak encrypted memory” from $\mathcal{A}$, ignore this query if keys have not yet been generated. Otherwise, send $(\text{corrupt-client}, 1)$ to $\mathcal{F}^{\text{BlindSpl}}$. Since both parties are now corrupted, our scheme does not provide any security guarantees under this setup.
- On command “Leak unencrypted memory” from $\mathcal{A}$, ignore this query if keys have not yet been generated. Otherwise, send $(\text{corrupt-client}, 2)$ to $\mathcal{F}^{\text{BlindSpl}}$. Since both parties are now corrupted, our scheme does not provide any security guarantees under this setup.
- On command “Corrupt client” from $\mathcal{A}$, send $(\text{corrupt-client}, 3)$ to $\mathcal{F}^{\text{BlindSpl}}$. If keys have been generated, proceed in the same way as when responding to “Leak unencrypted memory” command.

Signing: On command “Signing by client and adversary”, simulate the client’s protocol execution. Since the client is honest, we need to simulate protocol without knowing message msg :

- Send (sign-server, sid) to $\mathcal{F}^{\text{BlindSpl}}$ and get (sign-init, sid) as response.
- Generate random $\sigma_1 \leftarrow \mathbb{Z}_{n_1}$ and compute $m_* \leftarrow \sigma_1^e \bmod n_1$
- Set $y \leftarrow \sigma_1 \cdot m_*^{-d_1''} \bmod n_1$ and send $\langle y, m_*, w \rangle$ to $\mathcal{A}$.
- Upon receiving (σ_*, r', m_*) from $\mathcal{A}$, verify σ_*
- Generate random $\sigma \leftarrow \mathbb{Z}_n$ such that $M = \sigma^e \bmod n$ ends on $0xBC$
- Parse $M = \text{salt}_m \| H \| 0xBC$
- Sample $\text{mask} \leftarrow \{0, 1\}^{l_m}$ and compute $\text{salt} \leftarrow \text{mask} \oplus \text{salt}_m$. Program RO $\text{mask} \leftarrow H'(H, l_m)$.
- Send command (signature, sid, σ) to $\mathcal{F}^{\text{BlindSpl}}$ and save partial record (sid, $\perp_m$, σ , M , salt)

Now, we need to bind specific signature session i to a message msg . On a query to the random oracle H_1 of the form (msg, l) , respond as described on Figure 8.

// The simulated execution is different in the way values y, m_*, σ are generated. In the real protocol client's signature is calculated as $y \leftarrow (m_*)^{d_1'} \bmod n_1$, where $d_1' \leftarrow \text{genShare}(\text{PIN}, u)$. In the simulated protocol $y \leftarrow \sigma_1 \cdot m_*^{-d_1''} \bmod n_1$, where $\sigma_1 \leftarrow \mathbb{Z}_{n_1}$. Since message m_* in the real protocol is blinded as $m_* \leftarrow M \cdot r^e \bmod n_1$, where $r \leftarrow \mathbb{Z}_{n_1}^\times$, it is indistinguishable from randomly distributed message $m_* \leftarrow \sigma_1^e \bmod n_1$ for $\sigma_1 \leftarrow \mathbb{Z}_{n_1}$ in the simulated protocol. Thus, signatures are distributed identically. Additionally, programming RO should succeed as values H, mask are generated randomly during simulation. We need to program it during the signature generation, since H is outputted as part of the signature and adversary can query H' on this input.

Verification: Upon receiving message (verify, sid, $M = \text{msg}, \sigma$) from $\mathcal{F}^{\text{BlindSpl}}$:

- Verify signature using stored $\text{pk} = (e, n)$ following the instructions from Algorithm 1.
- If there is uncompleted record with (sid, $\perp_{\text{msg}}$, σ , M_i , salt_i), program RO as described on Figure 8. // uncompleted record means that σ was created during one of the "signing by client and adversary" interactions and the simulator does not know actual message that was signed msg . Hence, at this point we need to bind this signature session to a input message msg through programming RO.
- If there is no record with σ and the signature verifies, it means that this signature has been produced without simulator participating. However, RSA challenge is still embedded in the signature as the verification succeeds:
 - Find i such that $\sigma^e \bmod N \bmod n_1 \equiv w_i$. Fail if such i does not exist.
 - Since $w_i = y_i \cdot r_i^e \bmod n_1$, $y_i = (\sigma/r_i)^e$. Set $z_i \leftarrow \sigma/r_i$.
 - Save $(y_i, z_i) \in \mathbb{T}_{\text{RSA}}$.
 - Let q_R be the number of queries simulator made to the challenge oracle $\mathcal{O}_{\mathcal{R}}()$ and $q_I \leq q_R - 1$ be number of queries simulator made to the inversion oracle $\mathcal{O}_{\mathcal{I}}()$. If number of entries in $\mathbb{T}_{\text{RSA}}$ is q_R , output all $z_i \in \mathbb{T}_{\text{RSA}}$ to the external challenger as solution to RSA-ACTI problem, set $\phi \leftarrow 1$.

// For signatures created through the ideal functionality, output of the verification will be the same as in the real protocol. For signatures that adversary managed to create without making queries to the functionality, the output will be different. The simulated protocol execution returns 0 (as the key is not fully corrupted and the record has not been

created through the signing interface) and the real protocol execution returns 1. If adversary can distinguish the two worlds in the latter case, it should have solved RSA problem for the RSA challenge embedded through the random oracle. $\square$

6 Conclusion and Open Problems

We have introduced a new ideal functionality $\mathcal{F}^{\text{BlindSpl}}$, which captures security properties of blind server-supported signing. By combining the blindness properties of RSA Chaum-style blind signatures with efficiency of server-supported signatures, we have constructed Coppercloud protocol. It enables user to obtain a valid RSA signature on any message without revealing its content to the supporting server, while maintaining a distributed signing key between the user's device and the server.

As a future work, it would be interesting to construct blind server-supported protocols allowing to produce standardized signatures such as ECDSA, EdDSA and/or ML-DSA.

7 Acknowledgements

This work was funded by the Estonian Research Council under the grant number PRG1780.

Bibliography

- [1] Amjad, G., Yeo, K., Yung, M.: RSA blind signatures with public meta-data. Cryptology ePrint Archive, Paper 2023/1199 (2023), <https://eprint.iacr.org/2023/1199>
- [2] Asokan, N., Tsudik, G., Waidner, M.: Server-Supported Signatures. In: Bertino, E., Kurth, H., Martella, G., Montolivo, E. (eds.) Computer Security — ESORICS 96. pp. 131–143. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)
- [3] Bellare, Namprempe, Pointcheval, Semanko: The one-more-rsa-inversion problems and the security of chaum’s blind signature scheme. Journal of Cryptology **16**(3), 185–215 (Jun 2003). <https://doi.org/10.1007/s00145-002-0120-1>, <https://doi.org/10.1007/s00145-002-0120-1>
- [4] Bellare, M., Namprempe, C., Pointcheval, D., Semanko, M.: The power of rsa inversion oracles and the security of chaum’s rsa-based blind signature scheme. In: Syverson, P. (ed.) Financial Cryptography. pp. 319–338. Springer Berlin Heidelberg, Berlin, Heidelberg (2002). https://doi.org/10.1007/3-540-46088-8_25
- [5] Bellare, M., Rogaway, P.: Pss: Provably secure encoding method for digital signatures. IEEE P1363a: Provably secure signatures (1998), <https://api.semanticscholar.org/CorpusID:15110839>
- [6] Bicakci, K., Baykal, N.: Server assisted signatures revisited. In: Okamoto, T. (ed.) Topics in Cryptology – CT-RSA 2004. pp. 143–156. Springer Berlin Heidelberg, Berlin, Heidelberg (2004)
- [7] Bicakci, K., Baykal, N.: Improved server assisted signatures. Computer Networks **47**(3), 351–366 (2005). <https://doi.org/https://doi.org/10.1016/j.comnet.2004.08.008>, <https://www.sciencedirect.com/science/article/pii/S138912860400235X>
- [8] Buldas, A., Kalu, A., Laud, P., Oruaas, M.: Server-Supported RSA Signatures for Mobile Devices. In: Foley, S.N., Gollmann, D., Sneekenes, E. (eds.) Computer Security – ESORICS 2017. pp. 315–333. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-66402-6_19
- [9] Buldas, A., Laanoja, R., Truu, A.: A server-assisted hash-based signature scheme. In: Lipmaa, H., Mitrokotsa, A., Matulevičius, R. (eds.) Secure IT Systems. pp. 3–17. Springer International Publishing, Cham (2017)
- [10] Camenisch, J., Hohenberger, S., Lysyanskaya, A.: Compact E-Cash. In: Cramer, R. (ed.) Advances in Cryptology – EUROCRYPT 2005. pp. 302–321. Springer Berlin Heidelberg, Berlin, Heidelberg (2005). https://doi.org/10.1007/11426639_18
- [11] Camenisch, J., Lehmann, A., Neven, G., Samelin, K.: Virtual Smart Cards: How to Sign with a Password and a Server. In: Zikas, V., Prisco, R.D. (eds.) Security and Cryptography for Networks - 10th International Conference, SCN 2016, Amalfi, Italy, August 31 - September 2, 2016, Proceedings. Lec-

- ture Notes in Computer Science, vol. 9841, pp. 353–371. Springer (2016). https://doi.org/10.1007/978-3-319-44618-9_19
- [12] Canetti, R.: Universally composable signature, certification, and authentication. In: Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004. pp. 219–233 (2004). <https://doi.org/10.1109/CSFW.2004.1310743>
 - [13] Canetti, R.: Universally Composable Security. J. ACM **67**(5), 28:1–28:94 (2020), <https://doi.org/10.1145/3402457>
 - [14] Chaum, D.: Blind Signatures for Untraceable Payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) Advances in Cryptology. pp. 199–203. Springer US, Boston, MA (1983). https://doi.org/10.1007/978-1-4757-0602-4_18
 - [15] Cortier, V., Gaudry, P., Glondou, S.: Belenios: A Simple Private and Verifiable Electronic Voting System. In: Foundations of Security, Protocols, and Equational Reasoning: Essays Dedicated to Catherine A. Meadows, pp. 214–238. Springer (2019). https://doi.org/10.1007/978-3-030-19052-1_14
 - [16] Crites, E., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Snowblind: A Threshold Blind Signature in Pairing-Free Groups. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023. pp. 710–742. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-38557-5_23
 - [17] Curtmola, R., Garay, J., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: improved definitions and efficient constructions. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. p. 79–88. CCS ’06, Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1180405.1180417>
 - [18] Damgård, I., Mikkelsen, G.L., Skeltved, T.: On the Security of Distributed Multiprime RSA. In: Lee, J., Kim, J. (eds.) Information Security and Cryptology - ICISC 2014. pp. 18–33. Springer International Publishing, Cham (2015), https://doi.org/10.1007/978-3-319-15943-0_2
 - [19] Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy Pass: Bypassing Internet Challenges Anonymously. Proceedings on Privacy Enhancing Technologies **2018**(3), 164–180 (Apr 2018). <https://doi.org/10.1515/popets-2018-0026>
 - [20] Desmedt, Y., Frankel, Y.: Threshold cryptosystems. In: Brassard, G. (ed.) Advances in Cryptology — CRYPTO’ 89 Proceedings. pp. 307–315. Springer New York, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_28
 - [21] Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Secure Two-party Threshold ECDSA from ECDSA Assumptions. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 980–997. IEEE (2018), <https://doi.org/10.1109/SP.2018.00036>
 - [22] Fuchsbauer, G., Wolf, M.: Concurrently Secure Blind Schnorr Signatures. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024. pp. 124–160. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58723-8_5

- [23] Fujioka, A., Okamoto, T., Ohta, K.: A practical secret voting scheme for large scale elections. In: Seberry, J., Zheng, Y. (eds.) *Advances in Cryptology — AUSCRYPT '92*. pp. 244–251. Springer Berlin Heidelberg, Berlin, Heidelberg (1993). https://doi.org/10.1007/3-540-57220-1_66
- [24] Ganesan, R.: Yaksha: augmenting kerberos with public key cryptography. In: *Proceedings of the Symposium on Network and Distributed System Security*. pp. 132–143 (1995). <https://doi.org/10.1109/NDSS.1995.390639>
- [25] Garg, S., Rao, V., Sahai, A., Schröder, D., Unruh, D.: Round optimal blind signatures. In: Rogaway, P. (ed.) *Advances in Cryptology – CRYPTO 2011*. pp. 630–648. Springer Berlin Heidelberg, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22792-9_36
- [26] Gjosteen, K., Thuen, Ø.: Password-based signatures. In: Petkova-Nikova, S., Pashalidis, A., Pernul, G. (eds.) *Public Key Infrastructures, Services and Applications*. pp. 17–33. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
- [27] Hanzlik, L., Loss, J., Wagner, B.: Rai-Choo! Evolving Blind Signatures to the Next Level. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 753–783. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_26
- [28] Hazay, C., Katz, J., Koo, C.Y., Lindell, Y.: Concurrently-Secure Blind Signatures Without Random Oracles or Setup Assumptions. In: Vadhan, S.P. (ed.) *Theory of Cryptography*. pp. 323–341. Springer Berlin Heidelberg, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-70936-7_18
- [29] Heilman, E., Baldimtsi, F., Goldberg, S.: Blindly Signed Contracts: Anonymous On-Blockchain and Off-Blockchain Bitcoin Transactions. In: Clark, J., Meiklejohn, S., Ryan, P.Y., Wallach, D., Brenner, M., Rohloff, K. (eds.) *Financial Cryptography and Data Security*. pp. 43–60. Springer Berlin Heidelberg, Berlin, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53357-4_4
- [30] Henzinger, A., Dauterman, E., Corrigan-Gibbs, H., Zeldovich, N.: Private Web Search with Tiptoe. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. p. 396–416. SOSP '23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3600006.3613134>
- [31] Hinek, M.J.: On the security of multi-prime rsa. *Journal of Mathematical Cryptology* **2**(2), 117–147 (2008). <https://doi.org/10.1515/JMC.2008.006>
- [32] Jarecki, S., Nazarian, P.: Adaptively Secure Threshold Blind BLS Signatures and Threshold Oblivious PRF. *Cryptology ePrint Archive*, Paper 2025/483 (2025), <https://eprint.iacr.org/2025/483>
- [33] Kalu, A., Kus, B.C., Laud, P., Leung, K.L., Snetkov, N., Vakarjuk, J.: CACS: A Cloud Privacy-Preserving Attribute Management System. In: *Proceedings of the 18th International Conference on Availability, Reliability and Security. ARES '23*, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3600160.3605022>

- [34] Kamara, S., Moataz, T., Park, A., Qin, L.: A Decentralized and Encrypted National Gun Registry. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1520–1537 (2021). <https://doi.org/10.1109/SP40001.2021.00072>
- [35] Komlo, C., Goldberg, I.: FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In: Dunkelman, O., Jacobson, Jr., M.J., O’Flynn, C. (eds.) *Selected Areas in Cryptography*. pp. 34–65. Springer International Publishing, Cham (2021), https://doi.org/10.1007/978-3-030-81652-0_2
- [36] Kuchta, V., Manulis, M.: Rerandomizable Threshold Blind Signatures. In: Yung, M., Zhu, L., Yang, Y. (eds.) *Trusted Systems*. pp. 70–89. Springer International Publishing, Cham (2015). https://doi.org/10.1007/978-3-319-27998-5_5
- [37] Kumar, M., Katti, C.P., Saxena, P.C.: A Secure Anonymous E-Voting System Using Identity-Based Blind Signature Scheme. In: Shyamasundar, R.K., Singh, V., Vaidya, J. (eds.) *Information Systems Security*. pp. 29–49. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-72598-7_3
- [38] Lehmann, A., Nazarian, P., Özbay, C.: Stronger Security for Threshold Blind Signatures. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology – EUROCRYPT 2025*. pp. 335–364. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-91124-8_12
- [39] Lindell, Y.: Fast Secure Two-Party ECDSA Signing. *Journal of Cryptology* **34**(4) (Sep 2021), <http://dx.doi.org/10.1007/s00145-021-09409-9>
- [40] Lysyanskaya, A.: Security analysis of rsa-bssa. In: Boldyreva, A., Kolesnikov, V. (eds.) *Public-Key Cryptography – PKC 2023*. pp. 251–280. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-31368-4_10
- [41] Lysyanskaya, A., Ramzan, Z.: Group blind digital signatures: A scalable solution to electronic cash. In: Hirschfeld, R. (ed.) *Financial Cryptography*. pp. 184–197. Springer Berlin Heidelberg, Berlin, Heidelberg (1998). <https://doi.org/10.1007/BFb0055483>
- [42] Rivest, R.L., Adleman, L., Dertouzos, M.L., et al.: On Data Banks and Privacy Homomorphisms. *Foundations of secure computation* **4**(11), 169–180 (1978)
- [43] Shoup, V.: Practical Threshold Signatures. In: Preneel, B. (ed.) *Advances in Cryptology — EUROCRYPT 2000*. pp. 207–220. Springer Berlin Heidelberg, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-45539-6_15
- [44] Snetkov, N., Vakarjuk, J., Laud, P.: Universally Composable Server-Supported Signatures for Smartphones. *Cryptology ePrint Archive*, Paper 2024/1941 (2024), <https://eprint.iacr.org/2024/1941>
- [45] Sonnino, A., Al-Bassam, M., Bano, S., Meiklejohn, S., Danezis, G.: Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. In: *Proceedings 2019 Network and Distributed System Security Symposium. NDSS 2019*, Internet Society (2019). <https://doi.org/10.14722/ndss.2019.23272>

- [46] Sun, S.F., Au, M.H., Liu, J.K., Yuen, T.H.: RingCT 2.0: A Compact Accumulator-Based (Linkable Ring Signature) Protocol for Blockchain Cryptocurrency Monero. In: Foley, S.N., Gollmann, D., Sneekenes, E. (eds.) *Computer Security – ESORICS 2017*. pp. 456–474. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-66399-9_25
- [47] Thomson, M.: An Analysis of Apple’s Private Click Measurement (2022), <https://mozilla.github.io/ppa-docs/pcm.pdf>
- [48] Willemson, J.: Analyzing and Improving Eligibility Verifiability of the Proposed Belgian Remote Voting System. In: Rios, R., Posegga, J. (eds.) *Security and Trust Management*. pp. 126–135. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-47198-8_8
- [49] Xiao, Y., Du, J., Zhang, S., Zhang, W., Yang, Q., Zhang, D., Kifer, D.: Click Without Compromise: Online Advertising Measurement via Per User Differential Privacy. In: *2025 IEEE Symposium on Security and Privacy (SP)*. pp. 2919–2937 (2025). <https://doi.org/10.1109/SP61157.2025.00187>
- [50] Xu, S., Sandhu, R.: Two Efficient and Provably Secure Schemes for Server-Assisted Threshold Signatures. In: Joye, M. (ed.) *Topics in Cryptology — CT-RSA 2003*. pp. 355–373. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
- [51] Yi, X., Lam, K.Y.: A New Blind ECDSA Scheme for Bitcoin Transaction Anonymity. In: *Asia CCS ’19: Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*. p. 613–620. Asia CCS ’19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3321705.3329816>
- [52] Zohaib, A., Sheffey, J., Houmansadr, A.: Investigating Traffic Analysis Attacks on Apple iCloud Private Relay. In: *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*. p. 773–784. ASIA CCS ’23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3579856.3595793>